

Experiment ‘p_4_3_e_positions_of_vowels_array’ Results

June 3, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 2

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 2

Problem Specification

Problem name: p_4_3_e_positions_of_vowels_array

Natural language statement: Write a method that takes a string and returns the positions of all vowels in the string.

Method signature: p_4_3_e_positions_of_vowels_array(s: string) returns (positions: array<int>)

Ensures

- forall p :: p in positions[..] ==> 0 <= p < |s| && s[p] in ['a','e','i','o','u','A','E','I','O','U']
- forall i :: 0 <= i < |s| && s[i] in ['a','e','i','o','u','A','E','I','O','U'] ==> i in positions[..]

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that takes a string and returns the positions of all vowels in the string.

The signature should be:

`p_4_3_e_positions_of_vowels_array(s: string) returns (positions: array<int>)`

The method should respect the following contract:

ensures forall p :: p in positions[..] ==> 0 <= p < |s| && s[p] in ['a','e','i','o','u','A','E','I','O','U'], ensures forall i :: 0 <= i < |s| && s[i] in ['a','e','i','o','u','A','E','I','O','U'] ==> i in positions[..]

Produce and show only the Dafny body of this method, including the curly braces that surround it. Do not show the signature nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmfUl047FkWr9kCMYgQDoNUXaixCE

Model: gpt-5.5-2026-04-23

Created at: 1780491623

Finish reason: stop

Usage: 436 tokens in, and 4888 tokens out

```
{
  var vowels := ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'];
  var ps: seq<int> := [];
  var i := 0;

  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall p :: p in ps ==> 0 <= p < i && s[p] in vowels
    invariant forall j :: 0 <= j < i && s[j] in vowels ==> j in ps
  {
    if s[i] in vowels {
      var newPs := ps + [i];

      forall p | p in newPs
        ensures 0 <= p < i + 1 && s[p] in vowels
      {
        if p in ps {
          assert 0 <= p < i && s[p] in vowels;
        } else {
```

```

    assert p in [i];
    assert p == i;
  }
}

forall j | 0 <= j < i + 1 && s[j] in vowels
  ensures j in newPs
{
  if j < i {
    assert j in ps;
    assert j in newPs;
  } else {
    assert j == i;
    assert j in [i];
    assert j in newPs;
  }
}

ps := newPs;
} else {
  forall p | p in ps
    ensures 0 <= p < i + 1 && s[p] in vowels
  {
    assert 0 <= p < i && s[p] in vowels;
  }

  forall j | 0 <= j < i + 1 && s[j] in vowels
    ensures j in ps
  {
    if j < i {
      assert j in ps;
    } else {
      assert j == i;
      assert s[j] == s[i];
      assert false;
    }
  }
}

i := i + 1;
}

assert i == |s|;
positions := new int[|ps|](k => ps[k]);

assert positions.Length == |ps|;
forall k | 0 <= k < |ps|
  ensures positions[..][k] == ps[k]
{
  assert positions[k] == ps[k];
  assert positions[..][k] == positions[k];
}

```

```

}
assert |positions[..]| == |ps|;
assert positions[..] == ps;

assert vowels == ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'];
assert forall p :: p in positions[..] ==> 0 <= p < |s| && s[p] in ['a', 'e', 'i', 'o',
    ↪ 'u', 'A', 'E', 'I', 'O', 'U'];
assert forall j :: 0 <= j < |s| && s[j] in ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O
    ↪ ', 'U'] ==> j in positions[..];
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_4_3_e_positions_of_vowels_array/program.dfy(66,36): Error: index out of
    ↪ range
|
66 | positions := new int[|ps|](k => ps[k]);
    | ^

```

Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DmfWHCXqwl6B8KsoZTg36Mp3CPR3a

Model: gpt-5.5-2026-04-23

Created at: 1780491717

Finish reason: stop

Usage: 1259 tokens in, and 1795 tokens out

```

{
  var vowels := ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'];
  var ps: seq<int> := [];
  var i := 0;

  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall p :: p in ps ==> 0 <= p < i && s[p] in vowels
    invariant forall j :: 0 <= j < i && s[j] in vowels ==> j in ps
  {
    if s[i] in vowels {
      var newPs := ps + [i];

      forall p | p in newPs
    }
  }
}

```

```

    ensures 0 <= p < i + 1 && s[p] in vowels
  {
    if p in ps {
      assert 0 <= p < i && s[p] in vowels;
    } else {
      assert p in [i];
      assert p == i;
    }
  }
}

forall j | 0 <= j < i + 1 && s[j] in vowels
  ensures j in newPs
{
  if j < i {
    assert j in ps;
    assert j in newPs;
  } else {
    assert j == i;
    assert j in [i];
    assert j in newPs;
  }
}

ps := newPs;
} else {
  forall p | p in ps
    ensures 0 <= p < i + 1 && s[p] in vowels
  {
    assert 0 <= p < i && s[p] in vowels;
  }

  forall j | 0 <= j < i + 1 && s[j] in vowels
    ensures j in ps
  {
    if j < i {
      assert j in ps;
    } else {
      assert j == i;
      assert s[j] == s[i];
      assert false;
    }
  }
}

i := i + 1;
}

assert i == |s|;
positions := new int[|ps|](k => if 0 <= k && k < |ps| then ps[k] else 0);

assert positions.Length == |ps|;

```

```

forall k | 0 <= k < |ps|
  ensures positions[..][k] == ps[k]
{
  assert positions[k] == ps[k];
  assert positions[..][k] == positions[k];
}
assert |positions[..]| == |ps|;
assert positions[..] == ps;

assert vowels == ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'];
assert forall p :: p in positions[..] ==> 0 <= p < |s| && s[p] in ['a', 'e', 'i', 'o',
  ↪ 'u', 'A', 'E', 'I', 'O', 'U'];
assert forall j :: 0 <= j < |s| && s[j] in ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O
  ↪ ', 'U'] ==> j in positions[..];
}

```

Final Program

```

method p_4_3_e_positions_of_vowels_array(s: string) returns (positions: array<int>)
  ensures forall p :: p in positions[..] ==> 0 <= p < |s| && s[p] in ['a','e','i','o'
    ↪ ', 'u', 'A', 'E', 'I', 'O', 'U']
  ensures forall i :: 0 <= i < |s| && s[i] in ['a','e','i','o','u','A','E','I','O', '
    ↪ U'] ==> i in positions[..]
{
  var vowels := ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'];
  var ps: seq<int> := [];
  var i := 0;

  while i < |s|
    invariant 0 <= i <= |s|
    invariant forall p :: p in ps ==> 0 <= p < i && s[p] in vowels
    invariant forall j :: 0 <= j < i && s[j] in vowels ==> j in ps
    {
      if s[i] in vowels {
        var newPs := ps + [i];

        forall p | p in newPs
          ensures 0 <= p < i + 1 && s[p] in vowels
          {
            if p in ps {
              assert 0 <= p < i && s[p] in vowels;
            } else {
              assert p in [i];
              assert p == i;
            }
          }
        }

        forall j | 0 <= j < i + 1 && s[j] in vowels
          ensures j in newPs
          {

```

```

    if j < i {
        assert j in ps;
        assert j in newPs;
    } else {
        assert j == i;
        assert j in [i];
        assert j in newPs;
    }
}

ps := newPs;
} else {
    forall p | p in ps
        ensures 0 <= p < i + 1 && s[p] in vowels
    {
        assert 0 <= p < i && s[p] in vowels;
    }

    forall j | 0 <= j < i + 1 && s[j] in vowels
        ensures j in ps
    {
        if j < i {
            assert j in ps;
        } else {
            assert j == i;
            assert s[j] == s[i];
            assert false;
        }
    }
}

i := i + 1;
}

assert i == |s|;
positions := new int[|ps|](k => if 0 <= k && k < |ps| then ps[k] else 0);

assert positions.Length == |ps|;
forall k | 0 <= k < |ps|
    ensures positions[..][k] == ps[k]
{
    assert positions[k] == ps[k];
    assert positions[..][k] == positions[k];
}
assert |positions[..]| == |ps|;
assert positions[..] == ps;

assert vowels == ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U'];
assert forall p :: p in positions[..] ==> 0 <= p < |s| && s[p] in ['a', 'e', 'i', 'o',
    ↪ 'u', 'A', 'E', 'I', 'O', 'U'];

```

```
assert forall j :: 0 <= j < |s| && s[j] in ['a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O'  
    ↪ ', 'U'] ==> j in positions[..];  
}
```

Total Token Usage

Input tokens: 1695

Output tokens: 6683

Reasoning tokens: 5120

Sum of ‘total tokens’: 8378

Experiment Timings

Overall Experiment started at 1780491622888, ended at 1780491753105, lasting 130217ms (130.22 seconds)

Iteration #1 started at 1780491622888, ended at 1780491716910, lasting 94022ms (94.02 seconds)

Iteration #2 started at 1780491716911, ended at 1780491753105, lasting 36194ms (36.19 seconds)